

rauc-README

RAUC config — RK3588 A/B-virt emulator (PWU-AB-2)

Revision: 1 **Last modified:** 2026-06-11T00:00:00Z

Config artifacts the PWU-AB-2 RAUC dm-verity bundle build + in-guest rauc install consume. Authoritative design + reconciliation decision: [../..../docs/design/rk3588_ab_virt/PWU_AB_2_RAUC_VERITY.md](#). Test driver: [../ab_rauc_verity.sh](#).

Files

File	Role
system.conf	In-guest /etc/rauc/system.conf. bootloader=uboot; [slot.rootfs.0] bootname=A device=/dev/vda2, [slot.rootfs.1] bootname=B device=/dev/vda3 (the proven GPT layout). Installed into BOTH slot rootfs via a build_image.sh overlay (NOT wired yet).
fw_env.config	In-guest /etc/fw_env.config mapping fw_setenv/fw_printenv to the U-Boot env. UNVERIFIED — the raw offset is commented out until round-trip-verified against the built u- boot.bin CONFIG_ENV_* (the load- bearing gating dependency, shared with PWU-AB-4 §3).
manifest.raucm.in	RAUC bundle manifest TEMPLATE (format=verity). The build renders @VERSION@/@R00TFS_IMAGE@ and feeds it to rauc bundle.
gen_dev_keys.sh	Generates the throwaway self-signed dev signing key + cert into the gitignored ../out/rauc-keys/ (key chmod 600, NEVER committed,

File	Role
README.md	§11.4.10). The cert is the bundle signing cert AND the guest keyring. This document.

Status (§11.4.6 — honest)

- All files are **authored config artifacts** — parse-clean (`sh -n/bash -n` for the script) and internally coherent with the proven GPT layout + `BOOT_ORDER` scheme.
- **NOTHING here is a proven RAUC update.** No bundle has been built, no key generated, no `system.conf/fw_env.config` installed into a rootfs, no `rauc install` run. The reconciliation (extend `boot.cmd` to honor `BOOT_<bootname>_LEFT`) is **designed, not implemented** — see the design doc §2.
- The single biggest UNVERIFIED dependency is `fw_env.config` (the U-Boot env location) — see its inline header + design §4.5.

What the conductor must RUN next (NOT done here)

1. `gen_dev_keys.sh` (in the build container) → dev key+cert in `out/rauc-keys/`.
2. `Render manifest.raucm.in + rauc bundle ...` (design §4.3) → `out/images/update.raucb`.
3. Wire `system.conf + fw_env.config + u-boot-tools` into both slots via a `build_image.sh` overlay; **round-trip-verify fw_printenv/fw_setenv against U-Boot** (the gating check).
4. Run `../ab_rauc_verity.sh` with `RED_MODE=1` (capture defect-present), then flip `RED_MODE=0` once the apply switches slots.

Sources verified 2026-06-11

- RAUC integration (uboot backend, `system.conf`, `fw_env.config`): <https://rauc.readthedocs.io/en/latest/integration.html>
- RAUC examples (bundle build, dev cert, `format=verity` manifest): <https://rauc.readthedocs.io/en/latest/examples.html>
- RAUC `contrib/uboot.sh`: <https://raw.githubusercontent.com/rauc/rauc/master/contrib/uboot.sh>
- U-Boot RAUC Bootmeth: <https://docs.u-boot.org/en/latest/develop/bootstd/rauc.html>